



LOS CLUSTERS

Como plataforma de procesamiento paralelo

Autores:

Óscar Pino Morillas

Roberto Francisco Arroyo Moreno

Francisco Javier Nievas Muñoz

Índice

Introducción	2
Motivaciones de los clusters	3
Ventajas	3
Desventajas	4
Posibilidades	4
Arquitectura de los clusters	4
Clasificación de los clusters	6
La clase Beowulf	7
Herramientas software	8
Lenguajes / Compiladores	8
Entornos de programación / Sistemas Operativos	11
Equilibrado de cargas	12
Hardware de red	13
Ejemplos de clusters	14
Comparativas	15

Introducción

Existen diversas arquitecturas de ordenadores en paralelo. Estas se diferencian en como se organizan sus procesadores, memoria e interconexiones. Los sistemas más comunes son:

- MPP: Massive Parallel Processors
- SMP: Symmetric Multiprocessors
- CC-NUMA: Cache-Coherent Nonuniform Memory Access
- Sistemas distribuidos
- Clusters

Un MPP es un gran sistema de procesamiento paralelo con una arquitectura que típicamente esta formada de varios cientos de elementos de procesamiento que están interconectados a través de una red de interconexión de alta velocidad. Cada elemento suele disponer de una memoria principal y de uno o más procesadores (aunque incluso pueden disponer de periféricos como unidades de disco). En cada elemento (nodo) se ejecuta una copia separada del sistema operativo.

Un sistema SMP suele disponer de 2 a 64 procesadores y en su arquitectura todos los elementos son compartidos. En estos sistemas, todos los procesadores comparten todos los recursos disponibles. En estos sistemas sólo se ejecuta una copia del sistema operativo.

CC-NUMA es un sistema escalable multiprocesador que dispone de una memoria de acceso no uniforme. Al igual que los SMP, cada procesador en un CC-NUMA tiene una visión global de la memoria. Este tipo de sistema recibe su nombre (NUMA) debido a que el acceso a memoria requiere distintos tiempos de acceso según si la distancia a la que se encuentra la parte de memoria a la que se accede.

Un sistema distribuido no es más que una red convencional de ordenadores independientes. Cada máquina ejecuta su propio Sistema Operativo.

A un nivel básico un cluster (también llamado NOW-Network of Workstations o COW-Cluster of Workstations) es una colección de estaciones de trabajo o PC's que están interconectados mediante algún tipo de red. Generalmente estas redes son de alta velocidad. La principal característica de un cluster es que trabaja como una colección integrada de recursos y dispone de una única imagen del sistema que es compartida por todos sus nodos.

El uso de clusters para desarrollar, depurar y ejecutar aplicaciones paralelas está ganando en popularidad, convirtiéndose en una gran alternativa al uso de arquitecturas más especializadas debido al gran precio de estas. Un factor importante que ha hecho que el uso de clusters sea práctico es la estandarización de numerosas herramientas y utilidades usadas en aplicaciones paralelas. Ejemplos de estos estándares son la librería de paso de mensajes MPI (Message Passing Interface), y el lenguaje paralelo de datos High Performance Fortran (HPF). Esta estandarización permite que las aplicaciones sean desarrolladas y probadas (e incluso ejecutadas) en clusters, y a continuación, en una etapa posterior, puedan ser llevadas, con ligeras modificaciones, a plataformas paralelas especializadas.

Motivaciones

Ventajas

Cada uno de los nodos es una máquina completa que podremos utilizar como una estación de trabajo normal, al mismo tiempo que lo empleamos para nuestra máquina paralela.

Debido a que puede estar formado por PC's normales de gama media o incluso baja (Pentium, Pentium 2 o 3), su precio es relativamente bajo (causado por el gran mercado que hay actualmente).

Si no vamos a emplear cada una de los nodos como estación de trabajo, nos bastará con un teclado y una consola para todo el sistema, pudiéndonos evitar la compra incluso de la tarjeta de vídeo ya que el sistema operativo Linux (sobre el que se suele trabajar) puede ejecutarse sin todos estos elementos.

Como bien sabemos la construcción de un supercomputador de n procesadores (tomando n como un valor superior a 4) es complejo, caro y en muchos casos irrealizable. Sin embargo, crear una red con 16, 100 o incluso más nodos es, en comparación, algo trivial.

Cualquier problema en una máquina de este tipo es relativamente sencillo de resolver, ya que si por ejemplo se estropea un procesador en un nodo, nos bastará con sustituirlo por otro para resolverlo, esto es importante tenerlo en cuenta, ya que es infinitamente más sencillo encontrar un nuevo procesador Pentium que cualquiera de los componentes de un complejo supercomputador Fujitsu por ejemplo.

Las estaciones de trabajo están ganando en potencial de procesamiento. Su potencial se ha incrementado drásticamente en los últimos años, y se dobla cada 18-24 meses. Y todo indica a que esta tendencia continuará por varios años.

El ancho de banda de la comunicación entre estaciones de trabajo se está incrementando y el tiempo de latencia se decrecienta según se implementan las nuevas redes y protocolos en LAN.

Los clusters de estaciones de trabajo son más fáciles de integrar en redes existentes que ordenadores paralelos específicos.

Usualmente los usuarios de estaciones de trabajo no aprovechan toda la potencia de cómputo que estas les ofrecen.

Las herramientas de desarrollo para estaciones de trabajo están más "maduras" en comparación con las herramientas de desarrollo para ordenadores paralelos, esto se debe principalmente a la naturaleza no estándar de los distintos sistemas paralelos.

Los clusters de estaciones de trabajo son baratos y son una alternativa lista y disponible a las plataformas especializadas de computación paralela.

Los clusters son fácilmente escalables. La capacidad de los nodos se puede ampliar de forma sencilla, añadiendo memoria o procesadores adicionales.

Desventajas

Las redes de ordenadores, no están diseñadas para el procesamiento en paralelo (al menos no en principio), siendo su ancho de banda (cantidad de tráfico simultáneo) muchas veces demasiado bajo y su latencia (tiempo mínimo para transmitir un objeto) demasiado alta al menos en comparación con máquinas SMP.

No hay mucho software que sea capaz de tratar un cluster como si fuese un único sistema.

Aún así, y para finalizar, cabe decir a su favor que cada vez encontramos más software diseñado para aumentar el rendimiento de nuestro cluster con Linux, así como su facilidad de programación. Además, día a día, aparecen nuevas redes diseñadas específicamente para este tipo de arquitectura.

Posibilidades de los clusters

Procesamiento paralelo: Utilizar los múltiples procesadores para construir sistemas similares a MPP para procesamiento paralelo.

RAM en red: Utilizar la memoria asociada con cada estación de trabajo como una cache DRAM añadida. Esto puede mejorar drásticamente la memoria virtual y el manejo del sistema de archivos.

RAID (Redundant Array of Inexpensive Disks): Utilizar el conjunto de los discos duros de los PC's o estaciones de trabajo para proporcionar de forma barata y escalable almacenamiento de archivos seguro a través de la LAN.

Comunicación Multipath: Utilizar las distintas redes para transferencias en paralelo entre nodos.

Servidor de cálculo: Aprovechando la capacidad de procesamiento que nos ofrece un cluster podremos disponer de un servidor de cálculo barato y potente, formado por los mismos ordenadores sobre los que se trabaja habitualmente.

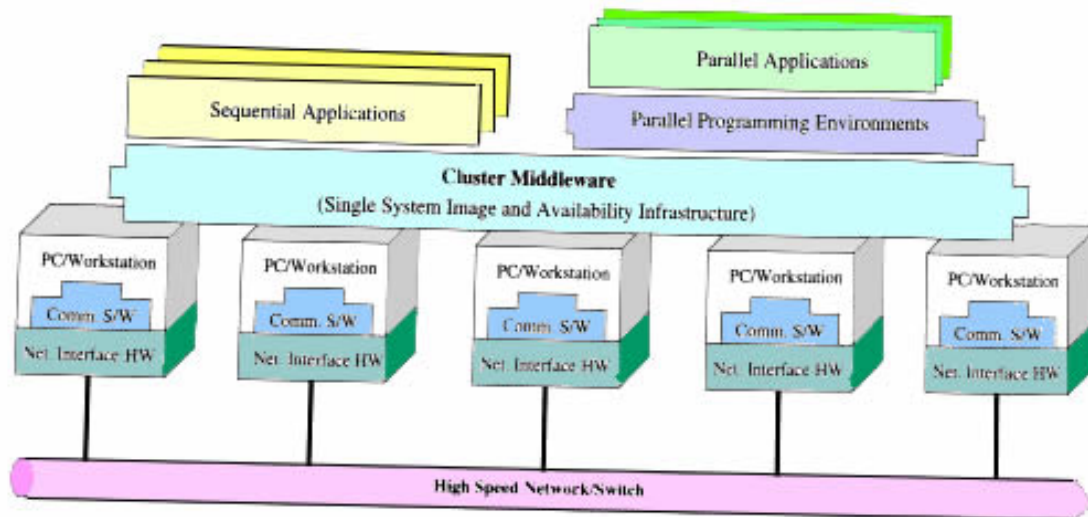
Arquitectura de los clusters

Un cluster es un tipo de sistema de procesamiento paralelo o distribuido, que consiste en una colección de PC's autónomos interconectados trabajando unidos como un solo recurso de computación integrado.

Un nodo del sistema puede ser un sistema con uno o más procesadores (PC's, estaciones de trabajo o SMP's) con memoria, recursos de entrada / salida, y un sistema operativo. Un cluster en general se refiere a dos o más ordenadores (nodos) conectados juntos. Estos nodos pueden coexistir en una misma caja, o bien estar físicamente separados y conectados a través de una LAN. Un cluster de ordenadores interconectados mediante una LAN puede parecer como un sistema único a los usuarios y las aplicaciones. Un sistema como este ofrece un mecanismo con buena

relación costo-prestaciones que permite obtener características que históricamente solo estaban disponibles en sistemas más caros.

Esta es la arquitectura típica de un cluster:



Comencemos a comentar el gráfico de abajo hacia arriba. Un cluster necesita:

- Una buena red de comunicaciones (como por ejemplo Gigabit Ethernet o Myrinet).
- Varios PC's, estaciones de trabajo o SMP's
- Tarjetas de interfaz a red (NIC's Network Interface Cards)
- Un sistema operativo especial
- Protocolos y Servicios para una comunicación rápida
- **Middleware** (explicado más adelante):
 - Hardware (como Digital Memory Channel, hardware DSM y técnicas SMP)
 - Kernel del Sistema operativo o Gluing Layer (como por ejemplo Solaris MC y GLUnix).
 - Aplicaciones y subsistemas (por ejemplo, el sistema de archivos paralelo)
- Entornos de programación paralela (como compiladores, PVM (Parallel Virtual Machine), y la librería MPI (Message Passing Interface)).
- Aplicaciones secuenciales y paralelas.

Middleware (SSI Single system Image y SAI System Availability Infrastructure):

Es una interfaz entre las aplicaciones y el hardware del cluster y el sistema operativo. Cada componente se encuentra a distinto nivel (capas):

- **SSI (Single System Image):**

Ilusión, creada por software o hardware, que hace parecer al cluster como una sola máquina para el usuario, las aplicaciones y la red (aporta transparencia).

Cada nodo puede acceder a un periférico o disco sin necesidad de saber su ubicación física.

Ejemplo:

telnet cluster.mi_servidor.es

telnet ~~nodo1~~.cluster.mi_servidor.es

- **SAI (System Availability Infrastructure):**

Esta capa proporciona al cluster servicios como:

- Checkpoints y recuperación de fallos.
- Tolerancia a fallos.

Clasificación de los clusters

Los clusters se pueden clasificar atendiendo a distintos aspectos:

1. Objetivo de la aplicación

Clusters de alto rendimiento (High Performance)
Clusters de alta disponibilidad

2. Uso de cada nodo

Clusters dedicados
Clusters no dedicados

En los primeros, se utilizan todos los recursos para la computación paralela, mientras q en los segundos, cada ordenador ejecuta sus propios programas y el cluster roba los ciclos en los que cada nodo no hace nada (que son muchos) y los utiliza para el procesamiento paralelo.

3. Hardware de cada nodo

Clusters de PC's (CoPs)
Clusters de estaciones de trabajo (COWs)
Clusters de SMP's (CLUMPs)

4. Sistema Operativo de los nodos

Linux (Ej. : Beowulf)
Solaris (Ej.: Berkeley NOW)
NT Clusters (Ej.: HPVM)
AIX Clusters (Ej.: IBM SP2)
Digital VMS Clusters
HP-UX clusters
Microsoft Wolfpack clusters

5. Configuración de los nodos

Clusters homogéneos: Todos los nodos tienen arquitecturas similares y ejecutan el mismo SO.
Clusters heterogéneos: Cada nodo tiene una arquitectura diferente y ejecuta un SO distinto.

6. Niveles de “clustering”

- Grupo (de 2 a 99 nodos)
- Departamento (de 10 a 100)
- Organización (mas de 100)
- Metacomputadores nacionales (WAN/Internet) (muchos nodos con clusters de departamento/organización)
- Metacomputadores internaciones (Internet) (desde miles hasta muchos millones de nodos)

La clase Beowulf

Aquí tenemos un tipo especial de cluster, con un costo inferior a los clusters convencionales y capaz de incrementar su rendimiento.

La clase Beowulf comenzó con un cluster en Linux diseñado en la NASA por Thomas Sterling y Don Becker en 1994. Poco a poco este proyecto creció dando lugar a un nuevo tipo de máquina.

Habiendo visto una descripción general de esta máquina además de algo de su historia, pasemos a analizar que es exactamente:

Un Beowulf es un conjunto de nodos minimalistas (con lo mínimo para funcionar), unidos por un medio de comunicaciones barato, por ejemplo una red Ethernet.

Al decir que son nodos minimalistas, nos referimos a que tienen lo mínimo para ejecutar su función, de hecho los nodos por sí solos son incapaces de ejecutar ni tan siquiera un sistema operativo.

Éstos constarán, en principio, de una placa base, una CPU, memoria y un dispositivo de comunicaciones, por lo general una tarjeta de red Ethernet o Fast Ethernet, también es posible que dispongan de un pequeño disco duro en el que arrancar el sistema operativo remoto, que será Linux para todos ellos.

Para la programación del Beowulf disponemos de varios métodos, pero primero debemos saber que está máquina no cuenta con mecanismos de memoria compartida, luego las comunicaciones se realizan siempre mediante pasos de mensajes (sockets, PVM (Parallel Virtual Machine)...).

Otra característica importante a tener en cuenta a la hora de la programación de este tipo de máquina es su topología física: red (en anillo, árbol...), así como los tipos de máquinas dispuestas en los nodos, ya que no tienen por que ser de potencia equivalente. Esto nos lleva a la conclusión de que un supercomputador clase Beowulf debería poseer una estructura lógica diseñada específicamente a su estructura física, ésta es quizá la mayor pega, ya que la programación y el diseño del mismo se nos complica con respecto a un Cluster normal, pero, aún así, el rendimiento obtenido empleando los mecanismos correctos, podrá ser superior.

La posible diferencia de potencia entre las distintas máquinas que constituyen el Beowulf plantea un **problema**: la necesidad de equilibrar la carga de trabajo entre los

distintos nodos, ya que si la distribución de carga fuese equitativa, los nodos de mayor potencia finalizarían antes su tarea y tendrían que esperar a los nodos de menor potencia, disminuyendo bastante la eficiencia.

Una aproximación de balanceo de carga es realizada por los usuarios a la hora de asignar los diferentes procesos de un trabajo paralelo a cada nodo, habiendo hecho una revisión previa de forma manual del estado de dichos nodos.

PVM y MPI, por ej., disponen de herramientas para la asignación inicial de procesos a cada nodo, sin considerar la carga existente en los mismos ni la disponibilidad de la memoria libre de cada uno. Estos paquetes corren a nivel de usuario como aplicaciones ordinarias, es decir son incapaces de activar otros recursos o de distribuir la carga de trabajo en el cluster dinámicamente. La mayoría de las veces es el propio usuario el responsable del manejo de los recursos en los nodos y de la ejecución manual de la distribución o migración de programas. Sin embargo, afortunadamente existen programas que realizan el equilibrado de carga de forma dinámica y sin que el usuario tenga que preocuparse de ello. Veremos algunos ejemplos más adelante.



Herramientas software

Lenguajes / Compiladores:

No encontraremos ningún lenguaje de programación para máquinas paralelas tan potente como el versátil **C** compilado con GCC. Aún así, existen lenguajes de más alto nivel adecuados para estos menesteres, entre ellos, cabe citar:

➤ **Fortran:**

Fortran, tal y como es hoy es un lenguaje de alto nivel, con numerosas mejoras con respecto al ANSI del 1966, (objetos como los de C++, instrucciones para procesamiento en paralelo,...)

De hecho podemos encontrar compiladores capaces de generar código para arquitecturas paralelas tipo SMP y Clusters entre otras. Como ejemplo de éstos: HPFC (HPF Compiler: <http://www.crpc.rice.edu/HPFF/>).

➤ **Mentat:**

Mentat es un lenguaje orientado a objetos para procesamiento en paralelo, funciona en máquinas clusters y se encuentra disponible para Linux. Su sintaxis es similar a la del popular C++.

➤ **mpC Programming Language**

mpC es un **lenguaje paralelo** de alto nivel (una extensión del C), diseñado especialmente para desarrollar aplicaciones adaptables y portables para redes heterogéneas de ordenadores. La idea principal subyacente en mpC es que cada aplicación define una red abstracta y distribuye datos, computación y comunicaciones sobre la red. El sistema de programación mpC usa esta información para reasignar la red abstracta sobre cualquier red real de forma que se asegura la eficiencia. Este “mapeado” se realiza en tiempo de ejecución y se basa en información acerca de la capacidad de los procesadores y los enlaces de la red real, adaptando dinámicamente el programa a la red.

El sistema mpC encapsula una plataforma particular de comunicación (actualmente, un subconjunto de MPI) asegurando la independencia de la plataforma del resto de componentes del sistema.

➤ **Cilk**

Cilk es un **lenguaje para programación** paralela multihebrada basado en C. Está diseñado para utilizarse como un lenguaje de propósito general, pero es específicamente efectivo para paralelismo asíncrono, que puede ser difícil de escribir utilizando un estilo de paso de mensajes. Cilk utiliza un “scheduler” que permite aproximar la eficiencia de forma precisa.

➤ **Jade/SAM (dialecto concurrente de C) sobre PVM:**

Jade es un **lenguaje de programación paralelo** (una extensión de C) para explotar la concurrencia en programas secuenciales de grano-grueso. Provee la conveniencia del modelo de memoria compartida permitiendo a cada tarea acceder a los objetos compartidos de forma transparente. Los programadores de Jade aumentan sus programas secuenciales con construcciones que descomponen la computación en tareas y declaran como esas tareas acceden a los objetos compartidos. La implementación de Jade interpreta de forma dinámica esta información para ejecutar el programa paralelo mientras preserva la semántica secuencial – si hay dependencia de datos entre tareas, las tareas se ejecutan en el mismo orden en el que lo harían en una ejecución secuencial. La estructura del paralelismo producida por un programa Jade es un grafo acíclico dirigido de tareas, donde cada arco entre nodos representa las restricciones de dependencia de los datos. Debido a que la construcción de la declaración del acceso a datos es ejecutada dinámicamente, este grafo de tareas puede ser dinámico y puede por tanto expresar la concurrencia disponible por la dependencia de datos solo en tiempo de ejecución.

➤ **Parallaxis (data parallel Modula-2) sobre PVM:**

Parallaxis es un **lenguaje de programación** estructurado para programación de datos paralelos (sistemas SIMD). El lenguaje está basado en Modula-2, pero ampliado por constructores paralelos independientes de la máquina. En Parallaxis, la abstracción se consigue declarando una configuración de procesador en forma funcional, especificando un número, el orden, y la conexión entre los procesadores.

Con estos 'procesadores y conexiones virtuales', un programa puede construirse independientemente del hardware real del computador.

➤ **pC++ (data parallel C++) sobre PVM:**

pC++ es un **C++ paralelo** portable para ordenadores de alto rendimiento. pC++ es una extensión de C++ que permite operaciones del estilo de datos-paralelos usando 'colecciones de objetos' desde alguna base elemental de clase. Las funciones miembro de esta clase elemental se pueden aplicar a la totalidad de la colección en paralelo. Esto permite a los programadores componer estructuras paralelas de datos con una semántica de ejecución paralela. Estas estructuras distribuidas pueden estar alineadas y distribuidas sobre la jerarquía de memoria de la máquina paralela tanto como en HPF. pC++ también incluye un mecanismo para el encapsulamiento al estilo de computación SPMD en un modelo de computación basado en hebras.

➤ **ZPL (lenguaje basado en vectores) sobre PVM:**

ZPL es un **lenguaje de programación** de vectores diseñado principalmente para una rápida ejecución en computadores secuenciales y paralelos. A causa de que ZPL se beneficia de la reciente investigación en compilación paralela, suministra un mecanismo de alto nivel para supercomputadores con una eficiencia comparable al paso de mensajes codificado "a pelo". Usuarios con experiencia en computación científica pueden aprender ZPL en pocas horas. Aquellos que hayan usado MATLAB o Fortran 90 tienen ya conocimiento del estilo de programación de vectores.

Características de ZPL:

ZPL es un lenguaje de vectores, por ello expresiones como $A+B$ hacen la suma de los vectores completos.

ZPL compila con ANSI C, que es compilado entonces con una librería dependiente de la máquina objetivo.

Si bien los programas deberían ser escritos enteramente en ZPL, el lenguaje puede unirse con código secuencial C o Fortran. De echo, ZPL provee acceso a librerías específicas.

Como ZPL es completamente portable, los programas que se desarrollen en una workstation se pueden recompilar para cualquier máquina paralela.

➤ **NESL**

NESL es un **lenguaje paralelo** que integra varias ideas de la comunidad teórica (algoritmos paralelos), de la comunidad de lenguajes (lenguajes funcionales) y de la comunidad de sistemas (muchas de las técnicas de implementación). Las ideas más importantes detrás de NESL son:

Paralelismo de datos anidados: esta característica ofrece los beneficios del paralelismo de datos, código conciso fácil de entender y depurar, mientras se adapta bien a algoritmos irregulares, como algoritmos en árboles, grafos o matrices dispersas.

Un lenguaje basado en el modelo de eficiencia: esto le da un mecanismo formal de calcular el trabajo y profundidad de un programa. Estas medidas están relacionadas con el tiempo de ejecución en máquinas paralelas.

El énfasis principal en el diseño de NESL fue hacer la programación paralela fácil y portable. Los algoritmos son normalmente significativamente más concisos en NESL que en la mayoría del resto de lenguajes. De hecho, el código se asemeja bastante a un pseudocódigo de alto nivel. Por supuesto, esto se consigue depositando mas

responsabilidad en el compilador y en el sistema de ejecución para lograr una buena eficiencia.

Por ejemplo, un algoritmo de quicksort en NESL seria:

```
function QUICKSORT(S) =  
  if (#S <= 1) then S  
  else  
    let a = S[rand(#S)];  
    S_1 = {e in S | e < a};  
    S_2 = {e in S | e == a};  
    S_3 = {e in S | e > a};  
    R = {QUICKSORT(v): v in [S_1, S_3]};  
    in R[0] ++ S_2 ++ R[1];  
  
quicksort([8, 14, -8, 5, -9, -3, 0, 17, 19]);
```

➤ Orca Parallel Programming Language

Orca es un **lenguaje de programación** paralela en sistemas distribuidos, basado en el modelo de objetos de datos compartido. Este modelo es una forma portable y simple del modelo basado en objetos de memoria distribuida compartida.

Entornos de programación/ Sistemas Operativos:

➤ MPI

MPI (Message Passing Interface). El objetivo básico de MPI es desarrollar un estándar de amplia utilización para escribir programas de paso de mensajes. Esta interfaz intenta establecer un práctico, portable, eficiente y flexible estándar para el paso de mensajes.

En el diseño del MPI se utilizó en gran medida el "MPI Forum" de forma que se tomaba en cuenta la opinión de los programadores. Ha sido influenciado en gran medida por el trabajo en el IBM T. J. Watson Research Center, Intel's NX/2, Express, nCUBE's Vertex, p4, y PARMACS. Otras contribuciones importantes provienen de Zipcode, Chimp, PVM, Chamaleon y PICL.

La principal ventaja de establecer un estándar en el paso de mensajes es la portabilidad y la facilidad de uso. En un sistema de memoria compartida en el cual las rutinas de más alto nivel y las abstracciones están construidas sobre una capa de bajo nivel de paso de mensajes los beneficios de la estandarización son particularmente aparentes. Además permite que se ofrezca soporte hardware y se aumente la escalabilidad.

➤ PVM:

La PVM (Parallel Virtual Machine) no es más que un API GNU de programación C para máquinas paralelas, capaz de crear una máquina virtual paralela, es decir, emplea los recursos libres de cada uno de los nodos sin tener que preocuparnos del cómo, haciéndonos creer que estamos ante una única supermáquina.

Un punto a destacar de PVM es que si disponemos de una red UNIX no tenemos más que instalar el software que, además, es libre y ya dispondremos de un

supercomputador paralelo con un coste nulo, y podremos ponernos a trabajar con él sin dejar de emplear cada máquina de la red para las mismas funciones que antes.

Además, PVM está portado para otras máquinas además de Linux, entre las que se encuentran distintos tipos de UNIX e incluso existe una versión para NT aunque cabe decir que alcanza unas velocidades bastante inferiores a las que alcanza con Linux.

➤ **Inferno:**

Inferno es un nuevo **sistema operativo** y un **entorno de programación** para repartir contenido en un rico entorno de redes heterogéneas, clientes y servidores. El sistema Inferno incluye el kernel Inferno, el lenguaje de programación Limbo, y APIs de referencia que incluyen interfaces para redes, gráficos, protocolos de red, seguridad y autenticación y varios kits de herramientas.

➤ **PGDBG:**

PGDBG es depurador para programas paralelos en MPI y en OpenMP pensado para ser utilizado en clusters con Linux.

Unas imágenes:



Equilibrado de cargas:

➤ **Mosix:**

Mosix es una herramienta desarrollada para sistemas tipo UNIX, cuya característica resaltante es el uso de algoritmos compartidos, los cuales están diseñados para responder al instante a las variaciones en los recursos disponibles, realizando el balanceo efectivo de la carga en el cluster mediante la migración automática de procesos o programas de un nodo a otro en forma sencilla y transparente.

El uso de Mosix en un cluster de PC's hace que éste trabaje de manera tal que los nodos funcionan como partes de un solo computador. El principal objetivo de esta herramienta es distribuir la carga generada por aplicaciones secuenciales o paralelizadas.

A diferencia de paquetes como MPI o PVM, Mosix realiza la localización automática de los recursos globales disponibles y ejecuta la migración dinámica "online" de procesos o programas para asegurar el aprovechamiento al máximo de cada nodo.

➤ **Condor:**

Condor es un manejador de carga del sistema especializado para trabajos de computación intensiva. Los usuarios envían sus trabajos a Condor, el cual los coloca en una cola, elige cuándo y dónde ejecutar los trabajos, y finalmente informa al usuario que se han completado.

Puede usarse para manejar un cluster dedicado de nodos de ordenadores (como los cluster "Beowulf").

Hardware de red

En las máquinas paralelas un elemento importante a tener en cuenta es la interfaz que utilizan para las comunicaciones, y antes de adquirir este debemos pararnos a pensar para qué queremos nuestro supercomputador paralelo, y el tráfico de datos que esperamos tener. Una vez tenido en cuenta esto, y atendiendo a nuestro presupuesto, seleccionaremos aquella interfaz que se ajuste más a lo deseado dentro de los posibles.

Podemos utilizar la siguiente tabla orientativa a la hora de esta elección. Hay que tener en cuenta que los datos pueden variar, especialmente el precio, ya que tan sólo está puesto como referencia y para comparativas:

Nombre	Ancho de banda	Latencia (ping)	Interfac e/Puerto	Tarj. red	Num. puestos	Switch / Hub	€ conexión /máquina
ATM	155 Mb/s	120 ms	PCI				
Ethernet	10 Mb/s	100 ms	PCI	9,62 €	8	129,79 €	16,23€
Fast Ethernet	100 Mb/s	80 ms	PCI	12,12 €	32	381,52 €	11,92€
Gigabit Ethernet	1000 Mb/s	300 ms	PCI	73,35 €	48	1547,15	32,23€
Myrinet	~2Gb/s	<7µs	PCI	995€-1,595€	16-128	1600€-12800€	100€
Token Ring	100 Mb/s-1Gb/s			170,82€	8	420.35€	52,54€
SCI	5 Gb/s	2,7 ms	PCI		8		

Precios actualizados al 10/11/2002

Ejemplos de clusters.

➤ ***The Stone SouperComputer (El Computador de la Sopa de piedras)***

Esta curiosa máquina ha sido construida con un costo igual a cero, gracias a donaciones individuales (como en el relato de la *Sopa de piedras*, de ahí su nombre). Está compuesta por máquinas que muchos considerarían ridículas: 486 DX/2 a 66Mhz, con de 16 a 32Mb RAM de media y discos duros de 400-600Mb, unidos por una red Ethernet (10Mbit/s).

A día de hoy (18/11/02), 133 máquinas, de las cuales:

- 53 Pentium (últimas incorporaciones).
- 5 Compac/DEC Alpha.

Emplean como sistema operativo RedHat Linux, y como lenguajes de programación, desde el GCC (GNU C Compiler) con la extensión PVM hasta Fortran.

Se ha empleado, y se emplea para entre otros trabajos:

- Análisis de los cambios en la distribución de las especies según los cambios del entorno.
- Un mapa nacional a gran resolución.
- La interpolación de 39 mapas que contienen las temperaturas mínimas y máximas (anuales/mensuales) obtenidas por más de 4000 estaciones meteorológicas. Cada uno de estos mapas contiene un total de 7,7 millones de celdas de 1 kilómetro cuadrado.

➤ ***Avalon:***

Es el Beowulf más famoso y más potente del mundo (de hecho sigue aún entre los 200 computadores más potentes del planeta) es *Avalon*, compuesto por 140 procesadores Alpha, 36 GB de RAM total, 500 GBs de disco duro, capaz de alcanzar los 48.5 Gflops (los flops son una medida de potencia que indica el número de operaciones de punto flotante que es capaz de ejecutar la máquina en un segundo).

El costo total de la máquina fue de unos 300.000 €.

Se encuentra situado en el Laboratorio de cálculo no-lineal de Los Álamos, y se emplea como supercomputador de propósito general abarca un rango de aplicaciones, entre ellas:

- Astrofísica.
- Dinámica molecular.
- Dinámica no-lineal y ecuaciones diferenciales.
- Fases de transición en el universo cercano.
- Simulaciones de líquidos y de cristales.



El supercomputador clase Beowulf "Avalon"

Comparativas

➤ **Benchmark con POV-RAY (persistence of vision raytracing)**

La tabla siguiente recoge la ejecución de un fichero "estándar" llamado skyvase.pov. Esta escena contiene reflexiones, mapeado de texturas y anti-aliasing para proporcionar un benchmark útil para comparar sistemas. Los test se ejecutan primero en el nodo maestro sólo, y luego se añaden los esclavos uno cada vez.



• **640x480**

num. de nodos	tiempo	ganancia
maestro solo	576.8	1.0
maestro + 1 esclavo	200.8	2.9
maestro + 2 esclavos	124.2	4.6
maestro + 3 esclavos	93.8	6.1
maestro + 4 esclavos	73.6	7.8

• **1024x768**

num. de nodos	tiempo	ganancia
maestro solo	1217.4	1.0
maestro + 1 esclavo	435.4	2.8
maestro + 2 esclavos	265.6	4.6
maestro + 3 esclavos	193.6	6.3
maestro + 4 esclavos	152.2	8.0

➤ **Odin Cluster vs. CRAY T3E**

La comparación de la eficiencia del cluster Beowulf "Odin I" con el Cray T3E 300MHz de NERSC muestra que el cluster es poco más del doble de rápido que el Cray TS3 cuando se usa el Compilador de Fortran del Portland Group. Cuando se usa el g77 (compilador de la distribución del LINUX), la eficiencia es también mejor que la del Cray T3E, pero no tan buena como la obtenida con el otro compilador. Con el compilador de Fortran gratuito de Pentium Group encontramos que la eficiencia es superior.

Una simulación molecular dinámica es la prueba que se realiza.

